

Machine Learning

“Learning denotes changes in the system that are adaptive in the sense that they enable the system to do the same task (or tasks drawn from the same population) more effectively the next time.” --

Herbert Simon

"Learning is constructing or modifying representations of what is being experienced." --Ryszard Michalski

"Learning is making useful changes in our minds." --Marvin Minsky

Learning is the process of acquiring new, or modifying existing, knowledge, behaviors, skills, values, or preferences. – Wikipedia

- An agent is learning if it improves its performance on future tasks after making observations about the world.
- Learning can range from the trivial, as exhibited by jotting down a phone number, to the profound, as exhibited by Albert Einstein, who inferred a new theory of the universe.

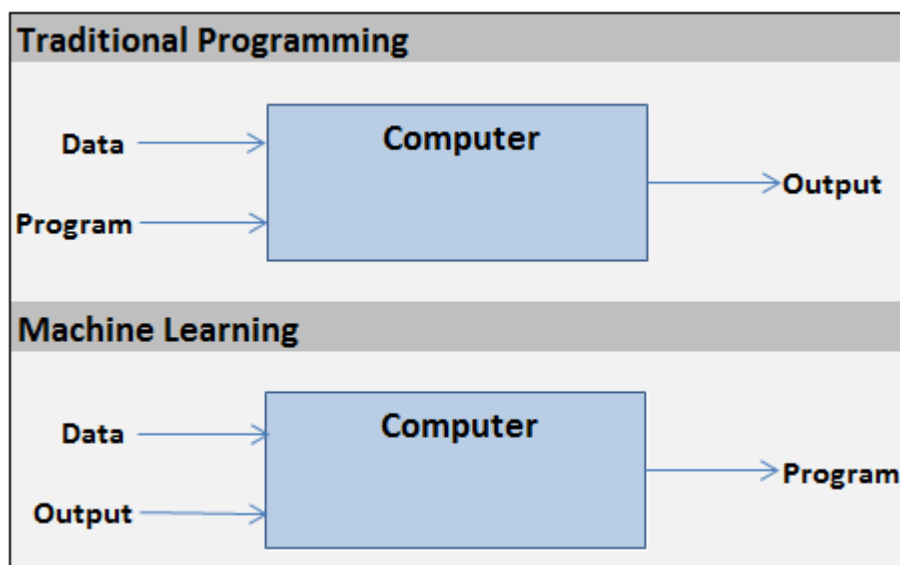
Why would we want an agent to learn? If the design of the agent can be improved, why wouldn't the designers just program in that improvement to begin with?

There are three main reasons.

1. First, the designers cannot anticipate all possible situations that the agent might find itself in. For example, a robot designed to navigate mazes must learn the layout of each new maze it encounters.
2. Second, the designers cannot anticipate all changes over time; a program designed to predict tomorrow's stock market prices must learn to adapt when conditions change from boom to bust.
3. Third, sometimes human programmers have no idea how to program a solution themselves. For example, most people are good at recognizing the faces of family members, but even the best programmers are unable to program a computer to accomplish that task, except by using learning algorithms.

What is machine learning?

- The term Machine Learning was coined by Arthur Samuel in 1959, an American pioneer in the field of computer gaming and artificial intelligence. According to him, “***Machine Learning is Field of study that gives computers the ability to learn without being explicitly programmed***”
- And in 1997, Tom Mitchell gave a “well-posed” mathematical and relational definition. According to him, “***A computer program is said to learn from experience E with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience E*** ”.
- The difference between traditional programming and machine learning can be illustrated with following figures (source: www.datasciencecentral.com)



Machine learning (ML) is a category of an algorithm that allows software applications to become more accurate in predicting outcomes without being explicitly programmed. The basic premise of

machine learning is to build algorithms that can receive input data and use statistical analysis to predict an output while updating outputs as new data becomes available.

Machine learning usually refers to the changes in systems that perform tasks associated with artificial intelligence (AI). Such tasks involve recognition, diagnosis, planning, robot control, prediction, etc. The changes might be either enhancements to already performing systems or synthesis of new systems

Any component of an agent can be improved by learning from data. The improvements, and the techniques used to make them, depend on four major factors:

- Which component is to be improved
- What prior knowledge the agent already has.
- What representation is used for the data and the component
- What feedback is available to learn from

Supervised, Unsupervised and Reinforcement Learning

- Depending upon types of feedback there are three main types of learning: Supervised, Unsupervised and Reinforcement

Supervised Learning

- In **supervised learning** the agent observes some example input–output pairs and learns a function that maps from input to output.
- The task of supervised learning is this:

Given a training set of N example input–output pairs

$$(x_1, y_1), (x_2, y_2), \dots (x_N, y_N),$$

where each y_j was generated by an unknown function $y = f(x)$,

discover a function h that approximates the true function f .

Here x and y can be any value; they need not be numbers. The function h is a hypothesis.

- Learning is a search through the space of possible hypotheses for one that will perform well, even on new examples beyond the training set.
- To measure the accuracy of a hypothesis we give it a test set of examples that are distinct from the training set. We say a hypothesis generalizes well if it correctly predicts the value of y for novel examples. Sometimes the function f is stochastic—it is not strictly a function of x , and what we have to learn is a conditional probability distribution, $P(Y | x)$.
- When the output y is one of a finite set of values (such as sunny, cloudy or rainy), the learning problem is called **classification**, and is called Boolean or binary classification if there are only two values.
- When y is a number (such as tomorrow's temperature), the learning problem is called **regression**. (Technically, solving a regression problem is finding a conditional expectation or average value of y , because the probability that we have found exactly the right real-valued number for y is 0.)

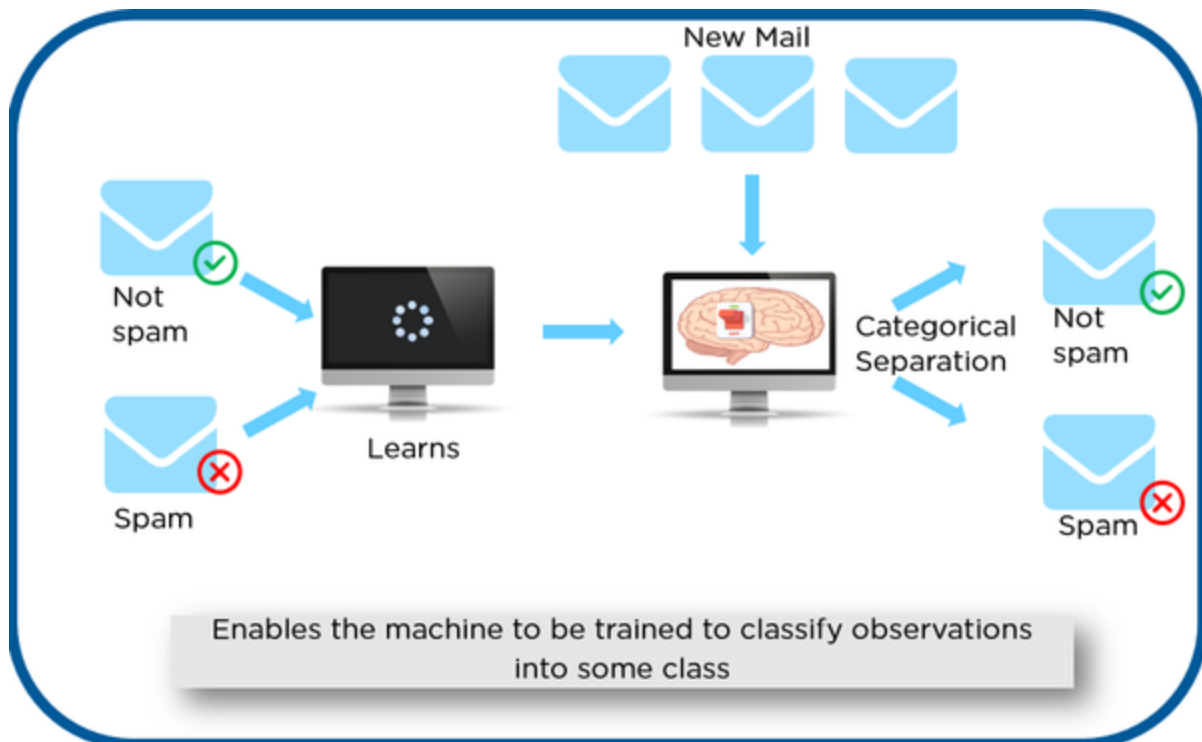
In Supervised learning, an AI system is presented with data which is labeled, which means that each data tagged with the correct label.

The goal is to approximate the mapping function so well that when you have new input data (x) that you can predict the output variables (Y) for that data.

- A model is prepared through a training process in which it is required to make predictions and is corrected when those predictions are wrong. The training process continues until the model achieves a desired level of accuracy on the training data.

It is called supervised learning because the process of an algorithm learning from the training dataset can be thought of as a teacher supervising the learning process. We know the correct answers, the algorithm iteratively makes predictions on the training data and is corrected by the teacher. Learning stops when the algorithm achieves an acceptable level of performance.

Example of Supervised Learning



As shown in the above example, we have initially taken some data and marked them as ‘Spam’ or ‘Not Spam’. This labeled data is used by the training supervised model, this data is used to train the model.

Once it is trained we can test our model by testing it with some test new mails and checking of the model is able to predict the right output.

Types of Supervised learning

- **Classification:** A classification problem is when the output variable is a category, such as “red” or “blue” or “disease” and “no disease”.
- **Regression:** A regression problem is when the output variable is a real value, such as “dollars” or “weight”.

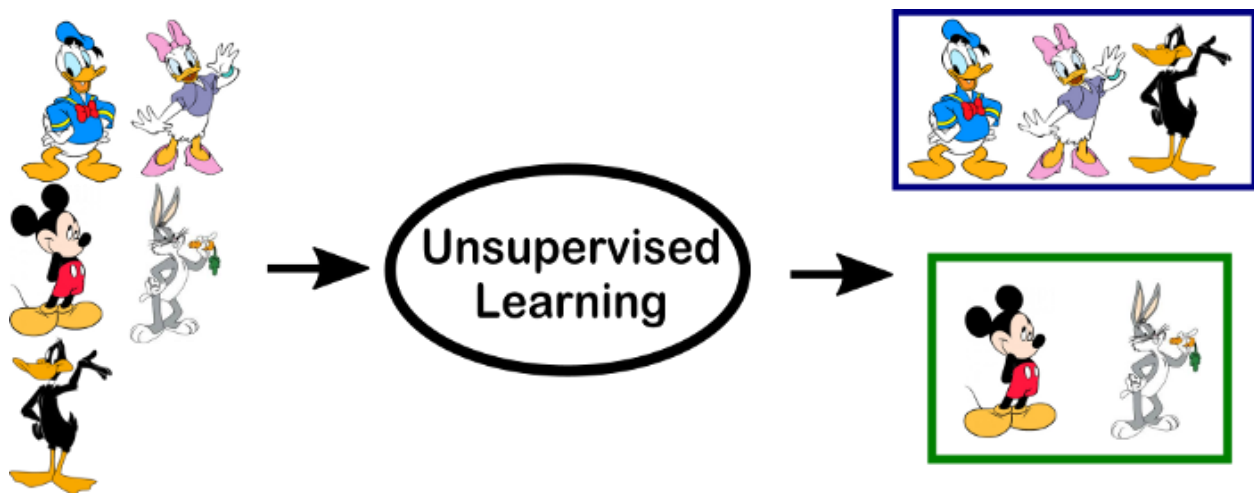
Supervised learning is the most mature, the most studied and the type of learning used by most machine learning algorithms. Learning with supervision is much easier than learning without supervision

Unsupervised Learning

- In **unsupervised learning** the agent learns patterns in the input even though no explicit feedback is supplied.
- The most common unsupervised learning task is **clustering**: detecting potentially useful clusters of input examples. For example, a taxi agent might gradually develop a concept of “good traffic days” and “bad traffic days” without ever being given labeled examples of each by a teacher.

In unsupervised learning, an AI system is presented with unlabeled, uncategorized data and the system’s algorithms act on the data without prior training. The output is dependent upon the coded algorithms. Subjecting a system to unsupervised learning is one way of testing AI.

Example of Unsupervised Learning



In the above example, we have given some characters to our model which are ‘Ducks’ and ‘Not Ducks’. In our training data, we don’t provide any label to the corresponding data. The unsupervised model is able to separate both the characters by looking at the type of data and models the underlying structure or distribution in the data in order to learn more about it.

- Input data is not labeled and does not have a known result.

- A model is prepared by deducing structures present in the input data. This may be to extract general rules. It may be through a mathematical process to systematically reduce redundancy, or it may be to organize data by similarity.
- It is called unsupervised learning because unlike supervised learning above there is no correct answers and there is no teacher. Algorithms are left to their own devices to discover and present the interesting structure in the data

Types of Unsupervised learning

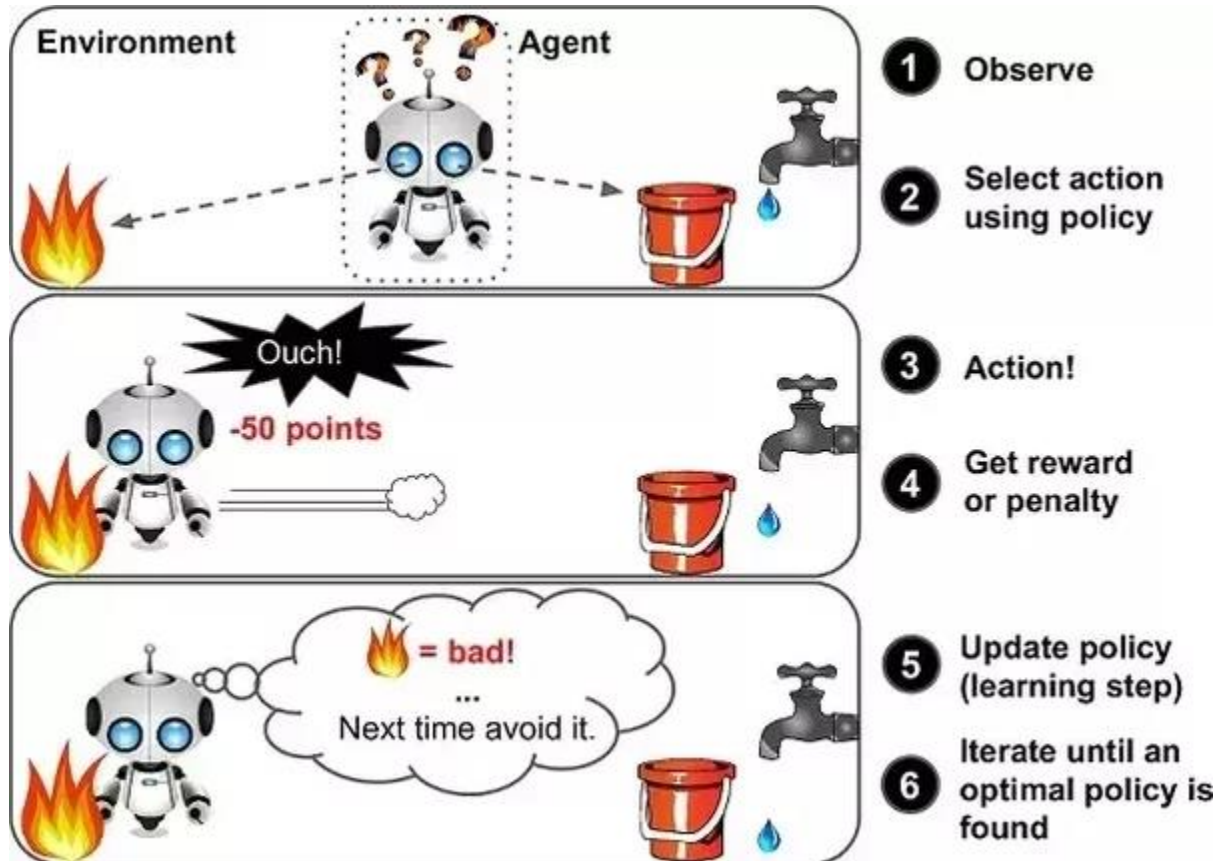
- **Clustering:** A clustering problem is where you want to discover the inherent groupings in the data, such as grouping customers by purchasing behavior.
- **Association:** An association rule learning problem is where you want to discover rules that describe large portions of your data, such as people that buy X also tend to buy Y.

Reinforcement learning

- In **reinforcement learning** the agent learns from a series of reinforcements—rewards or punishments. For example, the lack of a tip at the end of the journey gives the taxi agent an indication that it did something wrong. The two points for a win at the end of a chess game tells the agent it did something right. It is up to the agent to decide which of the actions prior to the reinforcement were most responsible for it.
- An agent can learn from success and failure, from reward and punishment.

A reinforcement learning algorithm, or agent, learns by interacting with its environment. The agent receives rewards by performing correctly and penalties for performing incorrectly. The agent learns without intervention from a human by maximizing its reward and minimizing its penalty. It is a type of dynamic programming that trains algorithms using a system of reward and punishment.

Example of Reinforcement Learning



In the above example, we can see that the agent is given two options i.e. a path with water or a path with fire. A reinforcement algorithm works on reward a system i.e. if the agent uses the fire path then the rewards are subtracted and agent tries to learn that it should avoid the fire path. If it had chosen the water path or the safe path then some points would have been added to the reward points, the agent then would try to learn what path is safe and what path isn't.

It is basically leveraging the rewards obtained, the agent improves its environment knowledge to select the next action

(Examples and some descriptions for supervised, unsupervised and reinforcement learning are taken from this source : <https://towardsdatascience.com/introduction-to-machine-learning-for-beginners-eed6024fdb08>)

Semi-Supervised Learning

- In **semi-supervised learning** we are given a few labeled examples and must make what we can of a large collection of unlabeled examples.
- Problems where you have a large amount of input data (X) and only some of the data is labeled (Y) are called semi-supervised learning problems.
- These problems sit in between both supervised and unsupervised learning.
- A good example is a photo archive where only some of the images are labeled, (e.g. dog, cat, person) and the majority are unlabeled.
- Many real world machine learning problems fall into this area. This is because it can be expensive or time-consuming to label data as it may require access to domain experts. Whereas unlabeled data is cheap and easy to collect and store.
- You can use unsupervised learning techniques to discover and learn the structure in the input variables.
- You can also use supervised learning techniques to make best guess predictions for the unlabeled data, feed that data back into the supervised learning algorithm as training data and use the model to make predictions on new unseen data.

Statistical-based Learning: Naive Bayes Model

- Statistical learning is a framework for machine learning based on statistics.
- Statistical learning is a framework for understanding data based on statistics, which can be classified as supervised or unsupervised. Supervised statistical learning involves building a statistical model for predicting, or estimating, an output based on one or more inputs, while in unsupervised statistical learning, there are inputs but no supervising output; but we can learn relationships and structure from such data.

Naive Bayes Model

- Statistical based learning model for classification
- Bayesian classifiers are statistical classifiers. They can predict class membership probabilities, such as the probability that a given tuple belongs to a particular class.

Bayesian classification is based on Bayes' theorem, named after Thomas Bayes (1702-1761).

- Naïve Bayesian classifiers assume that the effect of an attribute value on a given class is independent of the values of the other attributes. For example, a fruit may be considered to be an apple if it is red, round, and about 3 inches in diameter. Even if these features depend on each other or upon the existence of the other features, all of these properties independently contribute to the probability that this fruit is an apple and that is why it is known as 'Naive'.

Naive Bayes Algorithm is supervised classification algorithm which is based on Bayes theorem with an assumption of independence among features.

Bayes Theorem

- Let X be a data sample whose class label is unknown.
- Let H be some hypothesis: such that the data sample X belongs to a specific class C.
- We want to determine the probability that the hypothesis H holds given the observed data sample X (i.e. $P(H|X)$).
- $P(H|X)$ is the *posterior probability* representing our confidence in the hypothesis after X is given. In contrast, $P(H)$ is the *prior probability* of H for any sample, regardless of how the data in the sample look.
- The posterior probability $P(H|X)$ is based on more information than the prior probability $P(H)$.
- The Bayesian theorem provides a way of calculating the posterior probability $P(H|X)$ using probabilities $P(H)$, $P(X)$, and $P(X|H)$. The basic relation is:

$$P(H|X) = \frac{P(X|H) \times P(H)}{P(X)}$$

Or, the probability that an event H occurs given that another event X has already occurred is equal to the probability that the event X occurs given H has already occurred multiplied by probability that event H occurs divided by probability of occurrence of X.

For example, suppose our world of data tuples is confined to customers described by the attributes *age* and *income*, and that X is a 35-year-old customer with an income of \$40,000. Suppose that H is the hypothesis that our customer will buy a computer. Then,

- $P(H|X) \rightarrow$ the probability that customer X will buy a computer given that we know the customer's age and income. It is the posterior probability, or *a posteriori probability*, of H conditioned on X
- $P(X|H) \rightarrow$ the probability that a customer, X , is 35 years old and earns \$40,000, given that we know the customer will buy a computer. It is the posterior probability of X conditioned on H .
- $P(H) \rightarrow$ the probability that any given customer will buy a computer, regardless of age, income, or any other information. It is the prior probability, or *a priori probability*, of H .
- $P(X) \rightarrow$ the probability that a person from our set of customers is 35 years old and earns \$40,000. It is the prior probability of X .

Naïve Bayesian Classification

Let D be a training set of tuples and their associated class labels.

Given a tuple, X , the classifier will predict that X belongs to the class having the highest posterior probability, conditioned on X . That is, the naïve Bayesian classifier predicts that tuple X belongs to the class C_i if and only if

$$P(C_i|X) > P(C_j|X) \text{ for } 1 \leq j \leq m, j \neq i$$

Where,

$$P(C_i|X) = \frac{P(X|C_i) \times P(C_i)}{P(X)}$$

Here, $P(X)$ is constant for all classes. So, only $P(X|C_i) \times P(C_i)$ needs to be maximized.

It would be extremely computationally expensive to compute $P(X|C_i)$. In order to reduce computation in evaluating $P(X|C_i)$, the naïve assumption of class conditional independence is made. This presumes that the values of the attributes are conditionally independent of one another, given the class label of the tuple (i.e., that there are no dependence relationships among the attributes). Thus,

$$\begin{aligned} P(X|C_i) &= \prod_{k=1}^n P(X_k|C_i) \\ &= P(X_1|C_i) \times P(X_2|C_i) \times P(X_3|C_i) \times \cdots P(X_n|C_i) \end{aligned}$$

For example,

ID	Age	Income	Student	Credit_Rating	Buy_Computer?
1	Youth	High	No	Fair	No
2	Youth	High	No	Excellent	No
3	Middle_aged	High	No	Fair	Yes
4	Senior	Medium	No	Fair	Yes
5	Senior	Low	Yes	Fair	Yes
6	Senior	Low	Yes	Excellent	No
7	Middle_aged	Low	Yes	Excellent	Yes
8	Youth	Medium	No	Fair	No
9	Youth	Low	Yes	Fair	Yes
10	Senior	Medium	Yes	Fair	Yes
11	Youth	Medium	Yes	Excellent	Yes
12	Middle_aged	Medium	No	Excellent	Yes
13	Middle_aged	High	Yes	Fair	Yes
14	Senior	Medium	No	Excellent	No

Table : Buys_Computer data

Test data: X:(Age=Youth, Income=Medium, Student=Yes, Credit_Rating=Fair)

Let C_1 : Buys_Computer=Yes

C_2 : Buys_Computer=No

So,

$$P(C_1) = P(\text{Buys_Computer} = \text{Yes}) = 9/14 = 0.643$$

$$P(C_2) = P(\text{Buys_Computer} = \text{No}) = 5/14 = 0.357$$

To compute, $P(X|C_i)$ for $i=1,2$ we first compute following conditional probabilities:

$$P(\text{Age} = \text{Youth} | \text{Buys_Computer} = \text{Yes}) = 2/9 = 0.222$$

$$P(\text{Age} = \text{Youth} | \text{Buys_Computer} = \text{No}) = 3/5 = 0.600$$

$$P(\text{Income} = \text{Medium} | \text{Buys_Computer} = \text{Yes}) = 4/9 = 0.444$$

$$P(\text{Income} = \text{Medium} | \text{Buys_Computer} = \text{No}) = 2/5 = 0.400$$

$$P(\text{Student} = \text{Yes} | \text{Buys_Computer} = \text{Yes}) = 6/9 = 0.667$$

$$P(\text{Student} = \text{Yes} | \text{Buys_Computer} = \text{No}) = 1/5 = 0.200$$

$$P(\text{Credit_Rating} = \text{Fair} | \text{Buys_Computer} = \text{Yes}) = 6/9 = 0.667$$

$$P(\text{Credit_Rating} = \text{Fair} \mid \text{Buys_Computer} = \text{No}) = 2/5 = 0.400$$

Hence,

$$\begin{aligned} P(X|C_1) &= P(X \mid \text{Buys_Computer} = \text{Yes}) = P(\text{Age} = \text{Youth} \mid \text{Buys_Computer} = \text{Yes}) \times \\ &\quad P(\text{Income} = \text{Medium} \mid \text{Buys_Computer} = \text{Yes}) \times \\ &\quad P(\text{Student} = \text{Yes} \mid \text{Buys_Computer} = \text{Yes}) \times \\ &\quad P(\text{Credit_Rating} = \text{Fair} \mid \text{Buys_Computer} = \text{Yes}) \\ &= 0.222 \times 0.444 \times 0.667 \times 0.667 \\ &= 0.044 \end{aligned}$$

$$\begin{aligned} P(X|C_2) &= P(X \mid \text{Buys_Computer} = \text{No}) = P(\text{Age} = \text{Youth} \mid \text{Buys_Computer} = \text{No}) \times \\ &\quad P(\text{Income} = \text{Medium} \mid \text{Buys_Computer} = \text{No}) \times \\ &\quad P(\text{Student} = \text{Yes} \mid \text{Buys_Computer} = \text{No}) \times \\ &\quad P(\text{Credit_Rating} = \text{Fair} \mid \text{Buys_Computer} = \text{No}) \\ &= 0.600 \times 0.400 \times 0.200 \times 0.400 \\ &= 0.019 \end{aligned}$$

To find class C_i that maximizes $P(X|C_i)P(C_i)$, we compute,

$$P(X \mid \text{Buys_Computer} = \text{Yes}) \times P(\text{Buys_Computer} = \text{Yes}) = 0.444 \times 0.643 = 0.028$$

$$P(X \mid \text{Buys_Computer} = \text{No}) \times P(\text{Buys_Computer} = \text{No}) = 0.019 \times 0.357 = 0.007$$

Therefore Naïve Bayesian Classifier classifies

$X: (\text{Age} = \text{Youth}, \text{Income} = \text{Medium}, \text{Student} = \text{Yes}, \text{Credit_Rating} = \text{Fair})$

as class **Buys_Computer = Yes**

Another Example:

Following is a training data set of weather and corresponding target variable 'Play' (suggesting possibilities of playing). Now, we need to classify whether players will play or not based on weather condition. Let's follow the below steps to perform it.

Step 1: Convert the data set into a frequency table

Step 2: Create Likelihood table by finding the probabilities like Overcast probability = 0.29 and probability of playing is 0.64.

Let's understand it using an example. Below I have a training data set of weather and corresponding target variable 'Play' (suggesting possibilities of playing). Now, we need to classify whether players will play or not based on weather condition. Let's follow the below steps to perform it.

Step 1: Convert the data set into a frequency table

Step 2: Create Likelihood table by finding the probabilities like Overcast probability = 0.29 and probability of playing is 0.64.

Weather	Play
Sunny	No
Overcast	Yes
Rainy	Yes
Sunny	Yes
Sunny	Yes
Overcast	Yes
Rainy	No
Rainy	No
Sunny	Yes
Rainy	Yes
Sunny	No
Overcast	Yes
Overcast	Yes
Rainy	No

Frequency Table		
Weather	No	Yes
Overcast		4
Rainy	3	2
Sunny	2	3
Grand Total	5	9

Likelihood table				
Weather	No	Yes		
Overcast		4	=4/14	0.29
Rainy	3	2	=5/14	0.36
Sunny	2	3	=5/14	0.36
All	5	9		
	=5/14	=9/14		
	0.36	0.64		

Step 3: Now, use Naive Bayesian equation to calculate the posterior probability for each class. The class with the highest posterior probability is the outcome of prediction.

Problem: Players will play if weather is sunny. Is this statement is correct?

We can solve it using above discussed method of posterior probability.

$$P(\text{Yes} \mid \text{Sunny}) = P(\text{Sunny} \mid \text{Yes}) * P(\text{Yes}) / P(\text{Sunny})$$

Here we have $P(\text{Sunny} \mid \text{Yes}) = 3/9 = 0.33$, $P(\text{Sunny}) = 5/14 = 0.36$, $P(\text{Yes}) = 9/14 = 0.64$

Now, $P(\text{Yes} \mid \text{Sunny}) = 0.33 * 0.64 / 0.36 = 0.60$, which has higher probability.

Learning by Genetic Algorithms

- Nature has always been a great source of inspiration to all mankind. Genetic Algorithms (GAs) are search based algorithms based on the concepts of natural selection and genetics.
- GA reflects the process of natural selection where the fittest individuals are selected for reproduction in order to produce offspring of the next generation.
- GAs were developed by John Holland and his students and colleagues at the University of Michigan, most notably David E. Goldberg and has since been tried on various optimization problems with a high degree of success.
- It is frequently used to solve optimization problems
- Optimization problem is the problem of finding the best solution from all feasible solutions
- Optimization is the process of making something better. In any process, we have a set of inputs and a set of outputs.

Optimization refers to finding the values of inputs in such a way that we get the “best” output values. The definition of “best” varies from problem to problem, but in mathematical terms, it refers to **maximizing** or **minimizing** one or more objective functions, by varying the input parameters.

The set of all possible solutions or values which the inputs can take make up the search space. In this search space, lies a point or a set of points which gives the optimal solution. The aim of optimization is to find that point or set of points in the search space.

- GAs are a subset of a much larger branch of computation known as **Evolutionary Computation**.
- **Genetic algorithms simulate the process of natural selection** which means those species who can adapt to changes in their environment are able to survive and reproduce and go to next generation. In simple words, they simulate “survival of the fittest” among individual of consecutive generation for solving a problem. Each generation consist of a population of individuals and each individual represents a point in search space and possible solution.
- In GAs, we have a pool or a population of possible solutions to the given problem. These solutions then undergo recombination and mutation (like in natural genetics), producing

new children, and the process is repeated over various generations. Each individual (or candidate solution) is assigned a fitness value (based on its objective function value) and the fitter individuals are given a higher chance to mate and yield more “fitter” individuals. This is in line with the Darwinian Theory of “Survival of the Fittest”.

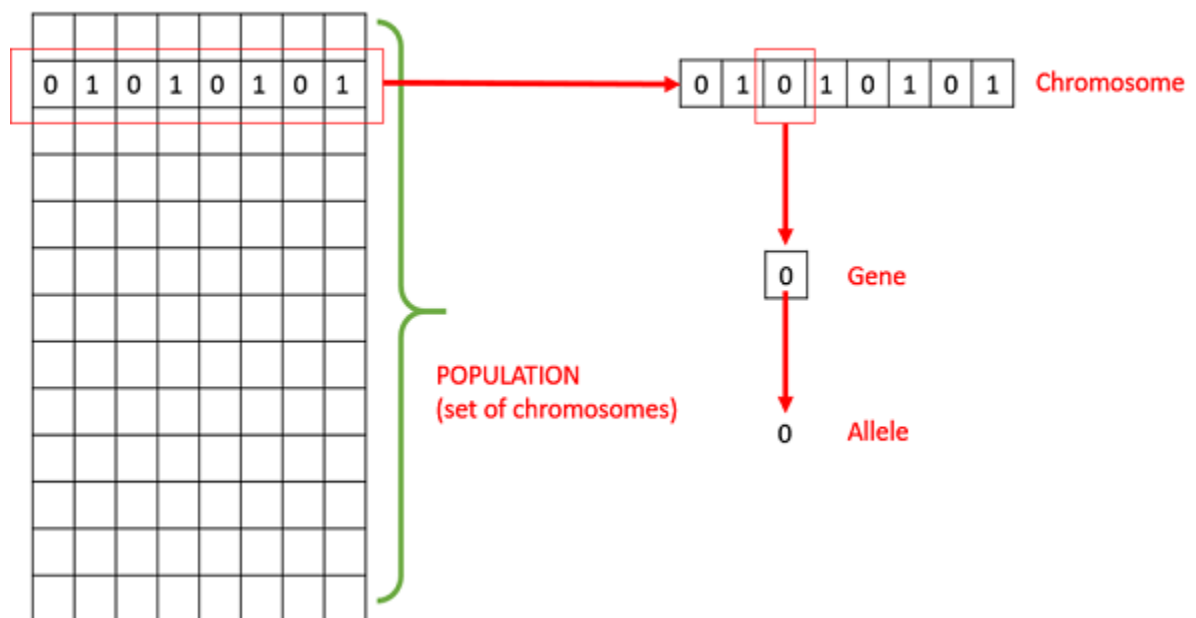
- In this way we keep “evolving” better individuals or solutions over generations, till we reach a stopping criterion.
- The Darwin theory of Evolution explain natural selection as
 - Individuals pass on traits to offspring
 - Individuals have different traits
 - Fittest individuals survive to produce more offspring
 - Over time, variation can accumulate leading to new species
- A genetic or evolutionary algorithm applies the principles of evolution found in nature to the problem of finding an optimal solution to a Solver problem. In a "genetic algorithm," the problem is encoded in a series of bit strings that are manipulated by the algorithm.

Basic Terminologies in GA

- **Population** – It is a subset of all the possible (encoded) solutions to the given problem. The population for a GA is analogous to the population for human beings except that instead of human beings, we have Candidate Solutions representing human beings.
- **Chromosomes** – A chromosome is one such solution to the given problem. A chromosome consists of genes, commonly referred as blocks of DNA, where each gene encodes a specific trait, for example hair color or eye color (in genetics).
- **Gene** – A gene is one element position of a chromosome.

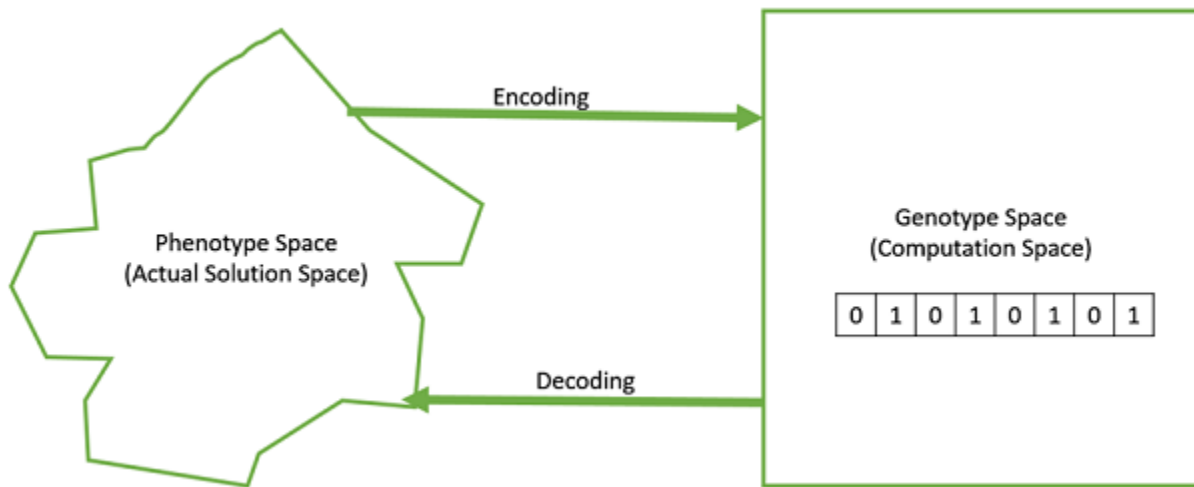
In a genetic algorithm, the set of genes of an individual is represented using a string, in terms of an alphabet. Usually, binary values are used (string of 1s and 0s). We say that we encode the genes in a chromosome.

- **Allele** – It is the value a gene takes for a particular chromosome.



Genotype – Genotype is the population in the computation space. In the computation space, the solutions are represented in a way which can be easily understood and manipulated using a computing system.

Phenotype – Phenotype is the population in the actual real world solution space in which solutions are represented in a way they are represented in real world situations.



Decoding and Encoding – For simple problems, the phenotype and genotype spaces are the same. However, in most of the cases, the phenotype and genotype spaces are different. Decoding is a process of transforming a solution from the genotype to the phenotype space, while encoding is a process of transforming from the phenotype to genotype space. Most common method of encoding is binary encoding. There are also other encoding techniques.

Fitness function

The fitness function determines how fit an individual is (the ability of an individual to compete with other individuals). It gives a fitness score to each individual. The probability that an individual will be selected for reproduction is based on its fitness score.

In some cases, the fitness function and the objective function may be the same, while in others it might be different based on the problem.

Genetic operators

A genetic operator is an operator used in genetic algorithms to guide the algorithm towards a solution to a given problem.

There are three main types of operators

1. Selection
2. Crossover
3. Mutation

Selection

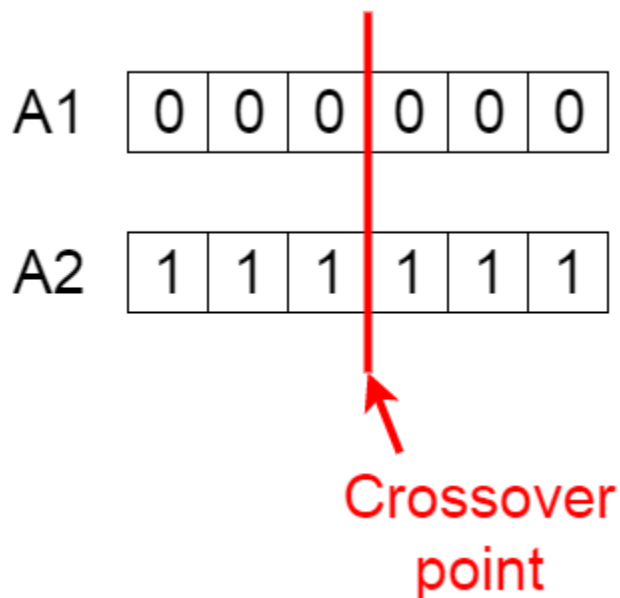
The idea of selection operator is to select the fittest individuals and let them pass their genes to the next generation.

Two pairs of individuals (parents) are selected based on their fitness scores. Individuals with high fitness have more chance to be selected for reproduction.

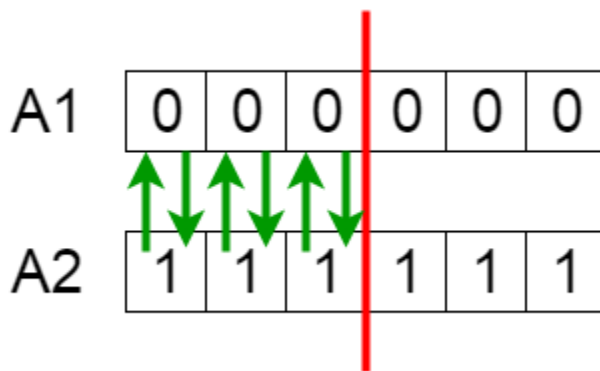
Crossover

Crossover is the most significant phase in a genetic algorithm. For each pair of parents to be mated, a crossover point is chosen at random from within the genes.

For example, consider the crossover point to be 3 as shown below.

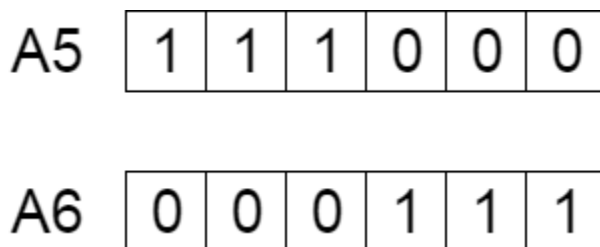


Offspring are created by exchanging the genes of parents among themselves until the crossover point is reached.



Exchanging genes among parents

The new offspring are added to the population.



New offsprings

Mutation

In certain new offspring formed, some of their genes can be subjected to a mutation with a low random probability. This implies that some of the bits in the bit string can be flipped.

Before Mutation

A5	1	1	1	0	0	0
----	---	---	---	---	---	---

After Mutation

A5	1	1	0	1	1	0
----	---	---	---	---	---	---

Mutation: Before and After

Mutation occurs to maintain diversity within the population and prevent premature convergence.

The Basic Structure of a Genetic Algorithm

GA starts with an initial population (which may be generated at random or seeded by other heuristics), select parents from this population for mating. Apply crossover and mutation operators on the parents to generate new off-springs. And finally these off-springs replace the existing individuals in the population and the process repeats. In this way genetic algorithms actually try to mimic the human evolution to some extent.

The algorithm terminates if the population has converged (does not produce offspring which are significantly different from the previous generation). Then it is said that the genetic algorithm has provided a set of solutions to our problem

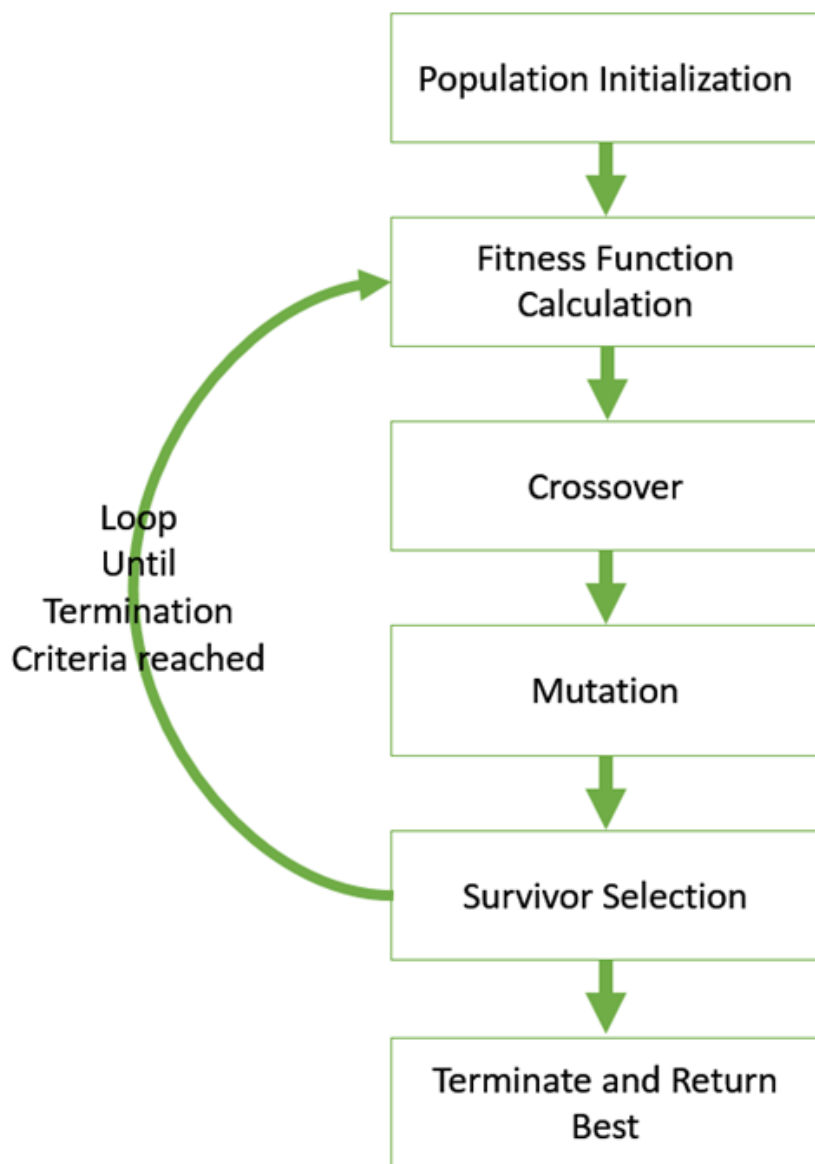


Fig: Basic flow in GA

Learning with Neural Networks

Neural networks are parallel computing devices, which is basically an attempt to make a computer model of the brain. The main objective is to develop a system to perform various computational tasks faster than the traditional systems. These tasks include pattern recognition and classification, approximation, optimization, and data clustering.

- A neuron is a cell in brain whose principle function is the collection, Processing, and dissemination of electrical signals. Brains Information processing capacity comes from networks of such neurons. Due to this reason some earliest AI work aimed to create such artificial networks. (Other Names are Connectionism; Parallel distributed processing and neural computing).

What is Artificial Neural Network?

(source: https://www.tutorialspoint.com/artificial_neural_network/artificial_neural_network_basic_concepts)

Artificial Neural Network (ANN) is an efficient computing system whose central theme is borrowed from the analogy of biological neural networks. ANNs are also named as “artificial neural systems,” or “parallel distributed processing systems,” or “connectionist systems.” ANN acquires a large collection of units that are interconnected in some pattern to allow communication between the units. These units, also referred to as nodes or neurons, are simple processors which operate in parallel.

Every neuron is connected with other neuron through a connection link. Each connection link is associated with a weight that has information about the input signal. This is the most useful information for neurons to solve a particular problem because the weight usually excites or inhibits the signal that is being communicated. Each neuron has an internal state, which is called an activation signal. Output signals, which are produced after combining the input signals and activation rule, may be sent to other units.

- We can think Artificial Neural Network as computational model that is inspired by the way biological neural networks in the human brain process information.

"...a computing system made up of a number of simple, highly interconnected processing elements, which process information by their dynamic state response to external inputs."

- Dr. Robert Hecht-Nielsen

Neural networks versus conventional computers

Neural networks take a different approach to problem solving than that of conventional computers. Conventional computers use an algorithmic approach i.e. the computer follows a set of instructions in order to solve a problem. Unless the specific steps that the computer needs to follow are known the computer cannot solve the problem. That restricts the problem solving capability of conventional computers to problems that we already understand and know how to solve. But computers would be so much more useful if they could do things that we don't exactly know how to do.

Neural networks process information in a similar way the human brain does. The network is composed of a large number of highly interconnected processing elements (neurones) working in parallel to solve a specific problem. Neural networks learn by example. They cannot be programmed to perform a specific task. The examples must be selected carefully otherwise useful time is wasted or even worse the network might be functioning incorrectly. The disadvantage is that because the network finds out how to solve the problem by itself, its operation can be unpredictable.

On the other hand, conventional computers use a cognitive approach to problem solving; the way the problem is to be solved must be known and stated in small unambiguous instructions. These instructions are then converted to a high level language program and then into machine code that the computer can understand. These machines are totally predictable; if anything goes wrong is due to a software or hardware fault.

Comparison between conventional computers and neural networks

Parallel processing

One of the major advantages of the neural network is its ability to do many things at once. With traditional computers, processing is sequential--one task, then the next, then the next, and so on. The idea of threading makes it appear to the human user that many things are happening at one time. For instance, the Netscape throbber is shooting meteors at the same time that the page is

loading. However, this is only an appearance; processes are not actually happening simultaneously.

The artificial neural network is an inherently multiprocessor-friendly architecture. Without much modification, it goes beyond one or even two processors of the von Neumann architecture. The artificial neural network is designed from the onset to be parallel. Humans can listen to music at the same time they do their homework--at least, that's what we try to convince our parents in high school. With a massively parallel architecture, the neural network can accomplish a lot in less time. The tradeoff is that processors have to be specifically designed for the neural network.

The ways in which they function

Another fundamental difference between traditional computers and artificial neural networks is the way in which they function. While computers function logically with a set of rules and calculations, artificial neural networks can function via images, pictures, and concepts.

Based upon the way they function, traditional computers have to learn by rules, while artificial neural networks learn by example, by doing something and then learning from it. Because of these fundamental differences, the applications to which we can tailor them are extremely different. We will explore some of the applications later in the presentation.

Self-programming

The "connections" or concepts learned by each type of architecture is different as well. The von Neumann computers are programmable by higher level languages like C or Java and then translating that down to the machine's assembly language. Because of their style of learning, artificial neural networks can, in essence, "program themselves." While the conventional computers must learn only by doing different sequences or steps in an algorithm, neural networks are continuously adaptable by truly altering their own programming. It could be said that conventional computers are limited by their parts, while neural networks can work to become more than the sum of their parts.

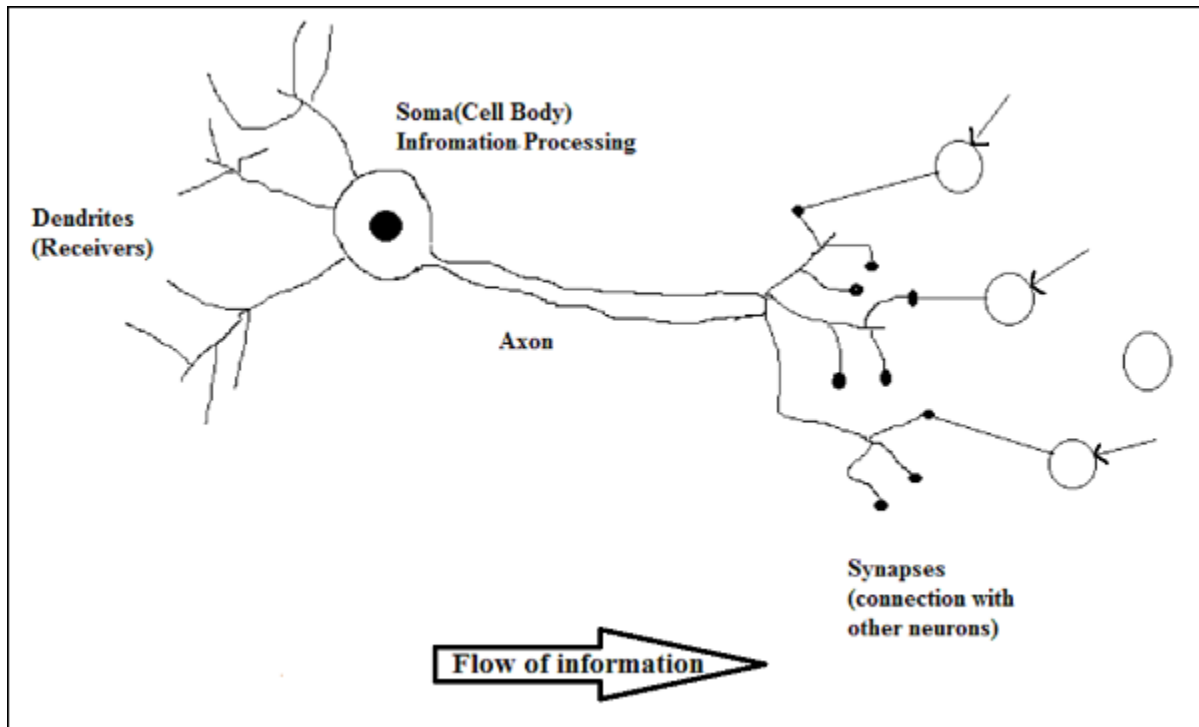
Speed

The speed of each computer is dependant upon different aspects of the processor. Von Neumann machines requires either big processors or the tedious, error-prone idea of parallel processors, while neural networks requires the use of multiple chips customly built for the application.

Biological Neural Networks Vs. Artificial Neural Networks (ANN)

Biological Neuron

A nerve cell (neuron) is a special biological cell that processes information. According to an estimation, there are huge number of neurons, approximately 10^{11} with numerous interconnections, approximately 10^{15}



Working of a Biological Neuron

As shown in the above diagram, a typical neuron consists of the following four parts with the help of which we can explain its working –

Dendrites – They are tree-like branches, responsible for receiving the information from other neurons it is connected to. In other sense, we can say that they are like the ears of neuron.

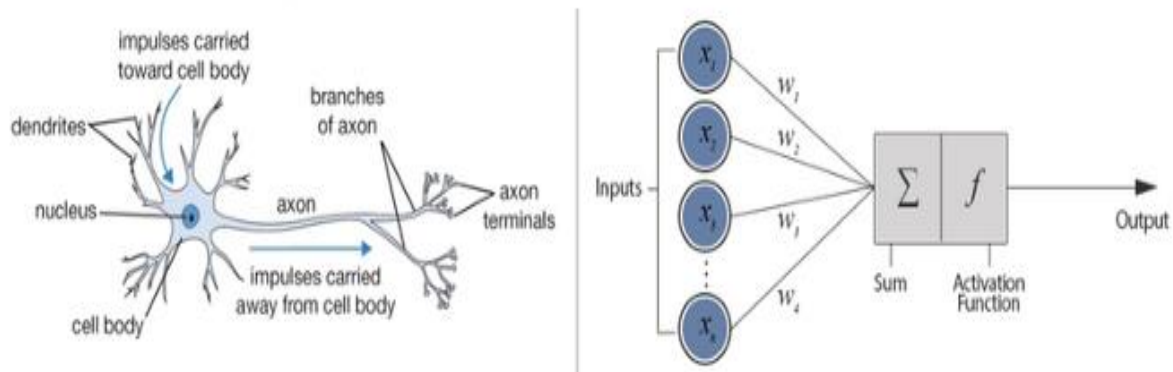
Soma – It is the cell body of the neuron and is responsible for processing of information, they have received from dendrites.

Axon – It is just like a cable through which neurons send the information.

Synapses – It is the connection between the axon and other neuron dendrites.

Biological Neural Network	Artificial Neural Network (ANN)
Soma	Node
Dendrites	Input
Synapse	Weights or Interconnections
Axon	Output

Biological Neuron versus Artificial Neural Network



Mathematical Model of ANN

Following figure shows a simple mathematical model of the neuron devised by McCulloch and Pitts (1943). Roughly speaking, it “fires” when a linear combination of its inputs exceeds some (hard or soft) threshold. A neural network is just a collection of units connected together; the properties of the network are determined by its topology and the properties of the “neurons.”

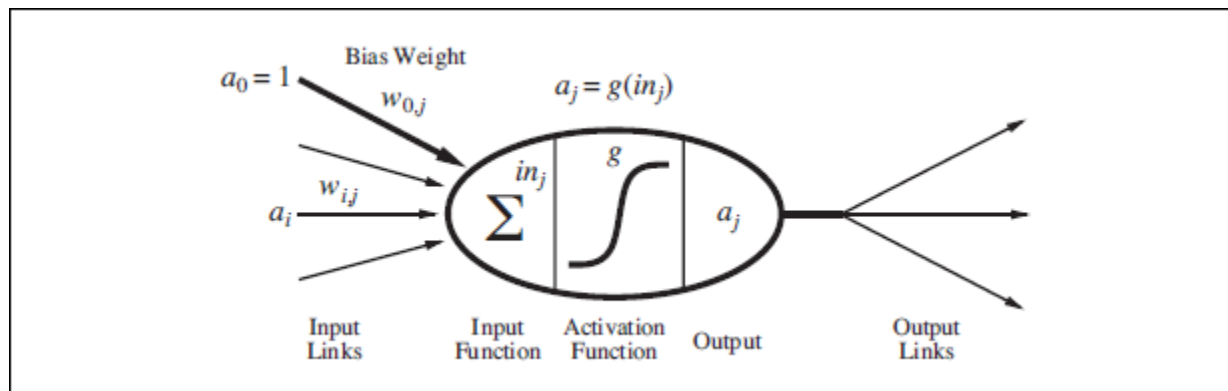


Figure: A simple mathematical model for a neuron. The unit’s output activation is $a_j = g(\sum_0^n w_{i,j} a_i)$ where , where a_i is the output activation of unit i and $w_{i,j}$ is the weight on the link from unit i to this unit.

Neural networks are composed of nodes or units connected by directed links. A link from unit i to unit j serves to propagate the activation a_i from i to j. Each link also has a numeric weight $w_{i,j}$ associated with it, which determines the strength and sign of the connection. Just as in linear regression models, each unit has a dummy input $a_0 = 1$ with an associated weight $w_{0,j}$. Each unit j first computes a weighted sum of its inputs:

$$in_j = \sum_0^n w_{i,j} a_i$$

Then it applies an activation function g to this sum to derive the output:

$$a_j = g(in_j) = g(\sum_0^n w_{i,j} a_i)$$

Activation Function

- Activation function decides, whether a neuron should be activated or not by calculating weighted sum of inputs (further adding bias with it) .
- Activation function is a function used to transform the activation level of a unit (neuron) into an output signal
- Activation function of a node defines the output of that node given an input or set of inputs
- Also called transfer function and squashing function
- Activation functions perform a transformation on the input received, in order to keep values within a manageable range.
- Activations functions falls under one of the following three categories :
 - Binary step functions
 - Linear functions
 - Non Linear functions

Binary step functions

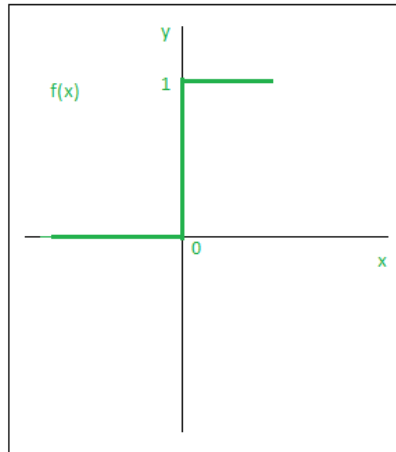
- A binary step function is a threshold-based activation function. If the input value is above or below a certain threshold, the neuron is activated and sends exactly the same signal to the next layer.
- The problem with a step function is that it does not allow multi-value outputs—for example, it cannot support classifying the inputs into one of several categories
- Step Function is one of the simplest kind of activation functions. In this, we consider a threshold value and if the value of net input say y is greater than the threshold then the neuron is activated.

Mathematically,

$$f(x) = \begin{cases} 0 & \text{if } x < k \\ 1 & \text{if } x \geq k \end{cases}$$

where k is threshold value

Given below is the graphical representation of step function.

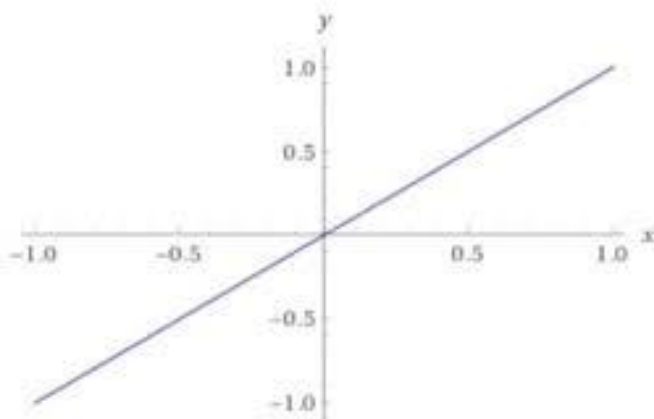


Linear activation function

- For linear activation (or identity) functions, the output is proportional to the total weighted input.
- General form :

$$f(x) = kx + c$$

Linear Activation Function



- Range : - infinity to + infinity
- It takes the inputs, multiplied by the weights for each neuron, and creates an output signal proportional to the input. In one sense, a linear function is better than a step function because it allows multiple outputs, not just yes and no
- However, a linear activation function has two major problems:

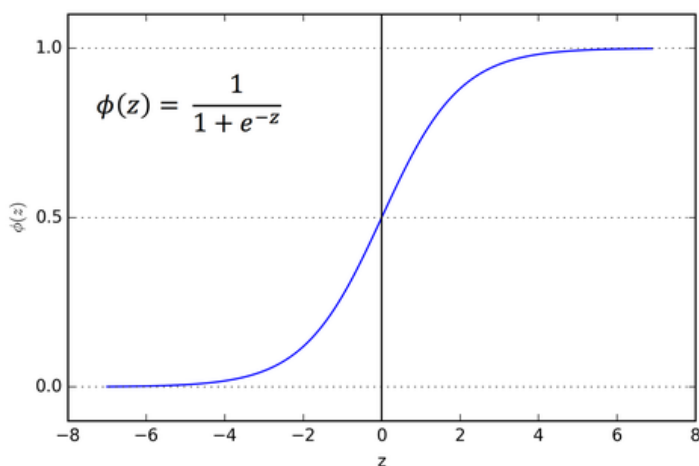
- Not possible to use backpropagation (gradient descent) to train the model—the derivative of the function is a constant, and has no relation to the input, X . So it's not possible to go back and understand which weights in the input neurons can provide a better prediction
- All layers of the neural network collapse into one—with linear activation functions, no matter how many layers in the neural network, the last layer will be a linear function of the first layer (because a linear combination of linear functions is still a linear function). So a linear activation function turns the neural network into just one layer.
- A neural network with a linear activation function is simply a linear regression model. It has limited power and ability to handle complexity varying parameters of input data.

Non Linear Activation Function

- Modern neural network models use non-linear activation functions. They allow the model to create complex mappings between the network's inputs and outputs, which are essential for learning and modeling complex data, such as images, video, audio, and data sets which are non-linear or have high dimensionality.
- Almost any process imaginable can be represented as a functional computation in a neural network, provided that the activation function is non-linear.
- Non-linear functions address the problems of a linear activation function:
 - They allow backpropagation because they have a derivative function which is related to the inputs.
 - They allow “stacking” of multiple layers of neurons to create a deep neural network. Multiple hidden layers of neurons are needed to learn complex data sets with high levels of accuracy
- Sigmoid, Hyperbolic Tan , ReLu etc are some examples

Sigmoid or Logistic Activation Function

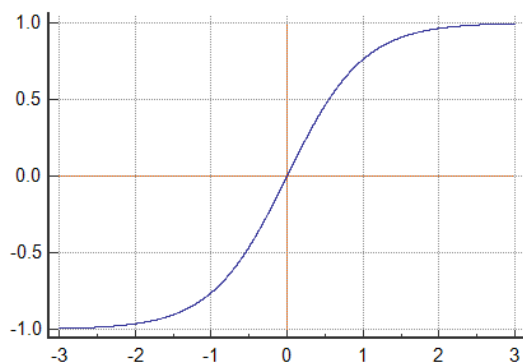
- The Sigmoid Function curve looks like a S-shape
- Range is between 0 to 1.



Tanh or hyperbolic tangent Activation Function

The range of the tanh function is from (-1 to 1). tanh is also sigmoidal (s - shaped).

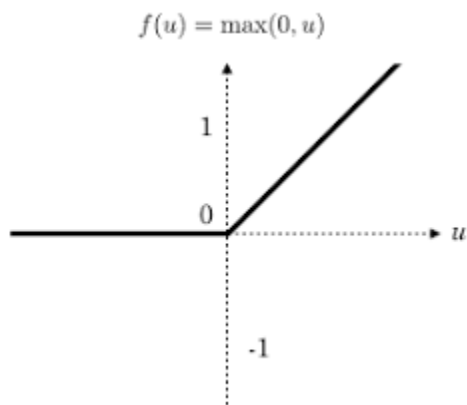
$$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$$



ReLU (Rectified Linear Unit) Activation Function

ReLU stands for rectified linear unit, and is a type of activation function. Mathematically, it is defined as

$$f(x) = \max(0, x)$$

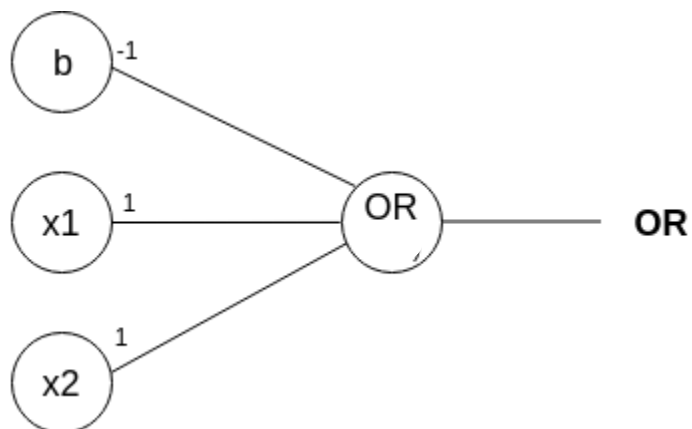


- Range : 0 to infinity

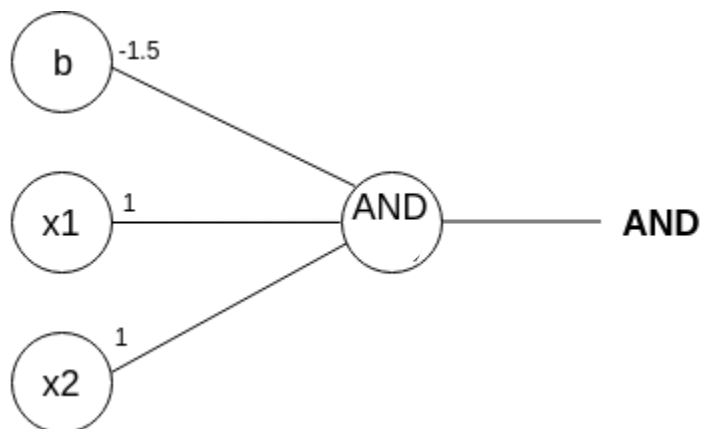
Realizing logic gates by using Neurons

Activation function $f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$

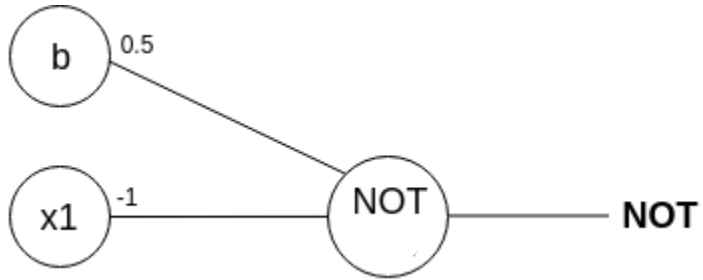
$w_1 = 1, w_2 = 1$ and $b = 1$



$b = -1.5, w_1 = 1, w_2 = 1$



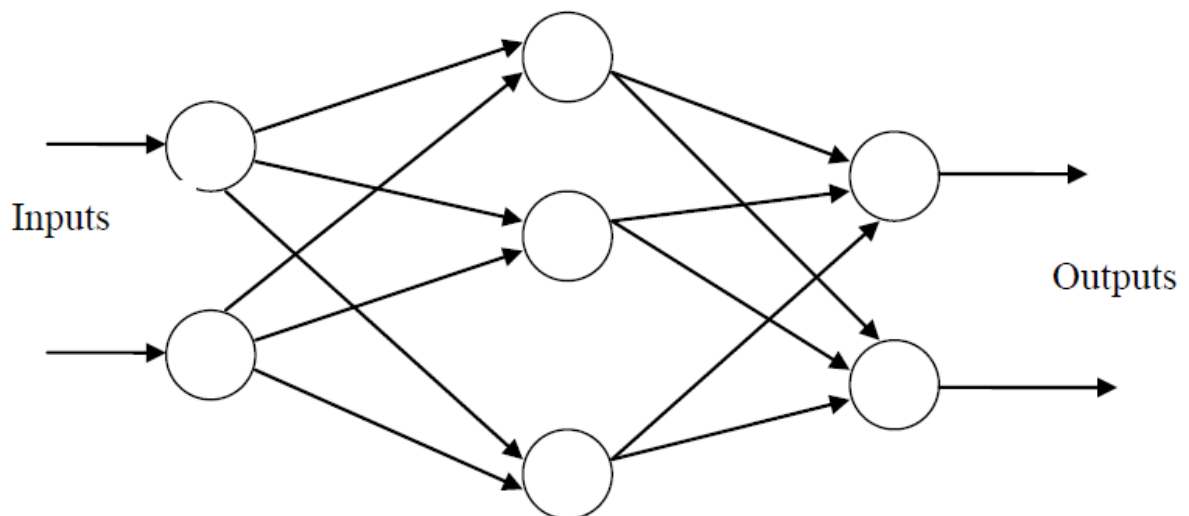
$b = 0.5, w_1 = -1$



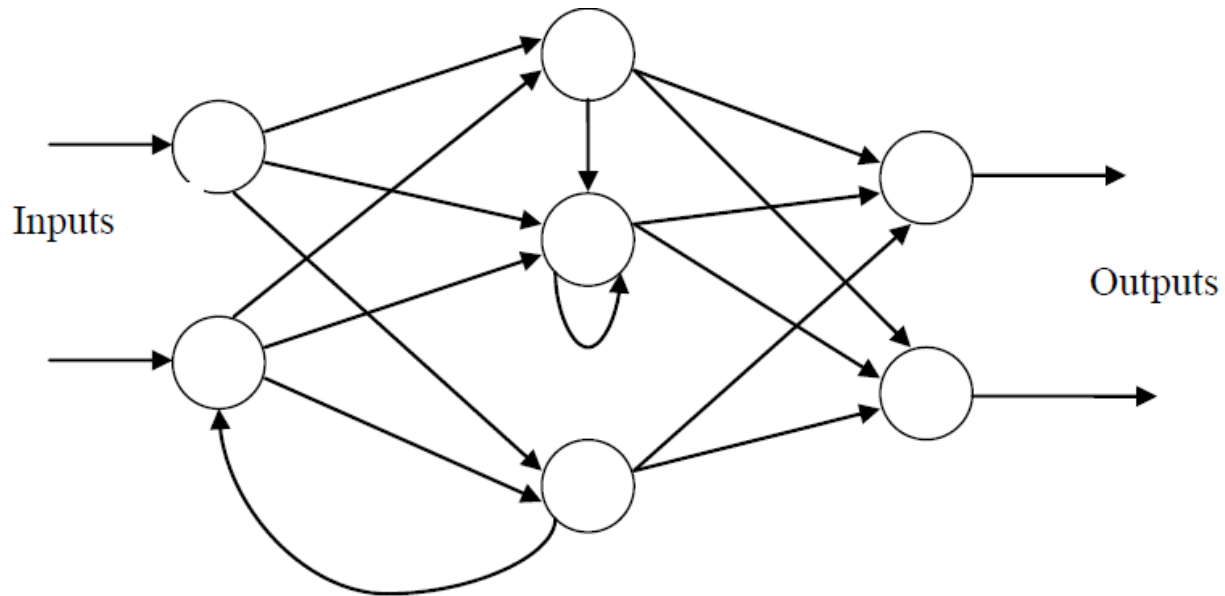
Network structures

Feed forward network

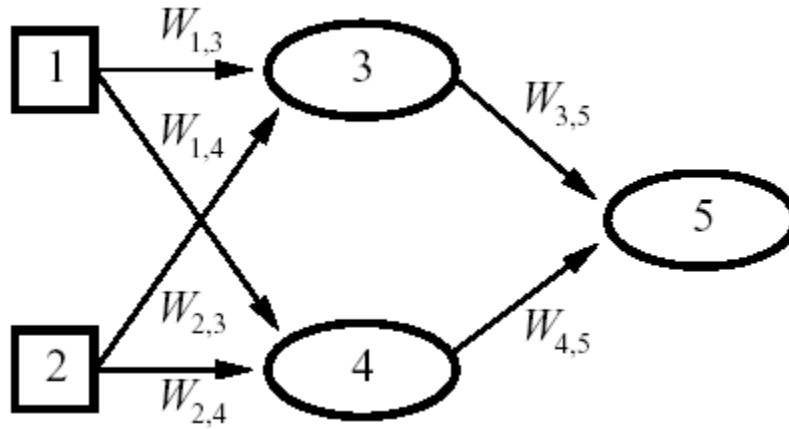
- Feed-forward ANNs allow signals to travel one way only; from input to output. There is no feedback (loops) i.e. the output of any layer does not affect that same layer. Feed-forward ANNs tend to be straight forward networks that associate inputs with outputs. They are extensively used in pattern recognition. This type of organization is also referred to as bottom-up or top-down.
- It is a non-recurrent network having processing units/nodes in layers and all the nodes in a layer are connected with the nodes of the previous layers. The connection has different weights upon them. There is no feedback loop means the signal can only flow in one direction, from input to output. It may be divided into the following two types –



Feedback networks (Recurrent networks)



- Feedback networks can have signals traveling in both directions by introducing loops in the network. Feedback networks are very powerful and can get extremely complicated. Feedback networks are dynamic; their 'state' is changing continuously until they reach an equilibrium point. They remain at the equilibrium point until the input changes and a new equilibrium needs to be found. Feedback architectures are also referred to as interactive or recurrent.
- As the name suggests, a feedback network has feedback paths, which means the signal can flow in both directions using loops. This makes it a non-linear dynamic system, which changes continuously until it reaches a state of equilibrium

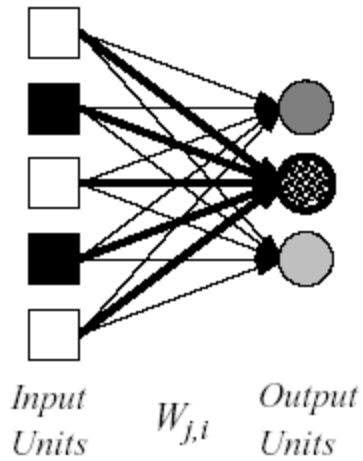
Feed-forward example

$$a_5 = g(W_{3,5} a_3 + W_{4,5} a_4)$$

$$= g(W_{3,5} g(W_{1,3} a_1 + W_{2,3} a_2) + W_{4,5} g(W_{1,4} a_1 + W_{2,4} a_2))$$

Types of Feed Forward Neural Network:**Single-layer neural networks (perceptrons)**

A neural network in which all the inputs connected directly to the outputs is called a single-layer neural network, or a perceptron network. Since each output unit is independent of the others each weight affects only one of the outputs.

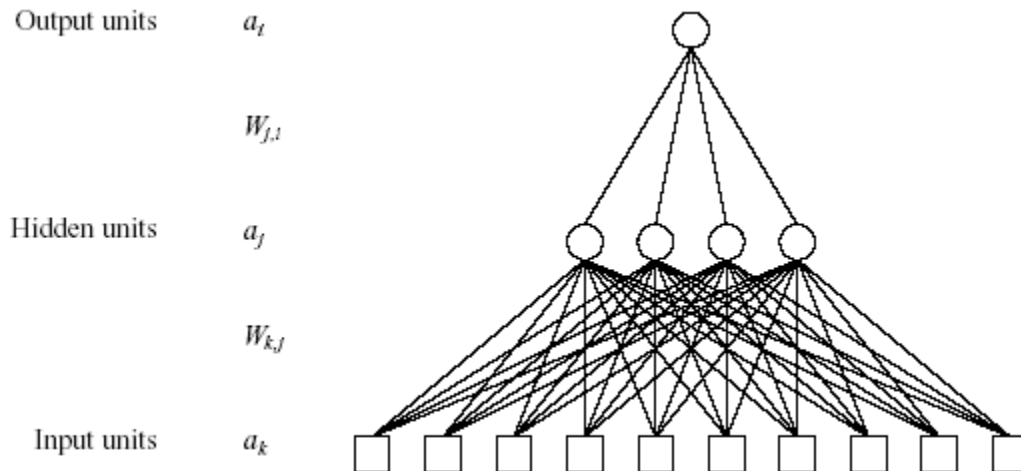


Multilayer neural networks (perceptrons)

The neural network which contains input layers, output layers and some hidden layers also is called multilayer neural network. The advantage of adding hidden layers is that it enlarges the space of hypothesis. Layers of the network are normally fully connected.

Once the number of layers, and number of units in each layer, has been selected, training is used to set the network's weights and thresholds so as to minimize the prediction error made by the network

Training is the process of adjusting weights and threshold to produce the desired result for different set of data.



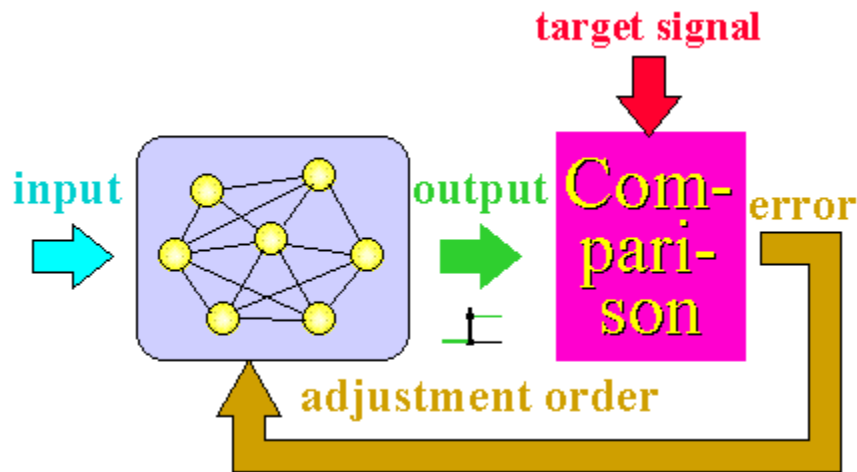
Learning in Neural Networks

Learning: One of the powerful features of neural networks is learning. Learning in neural networks is carried out by adjusting the connection weights among neurons. It is similar to a biological nervous system in which learning is carried out by changing synapses connection strengths, among cells.

The operation of a neural network is determined by the values of the interconnection weights. There is no algorithm that determines how the weights should be assigned in order to solve specific problems. Hence, the weights are determined by a learning process

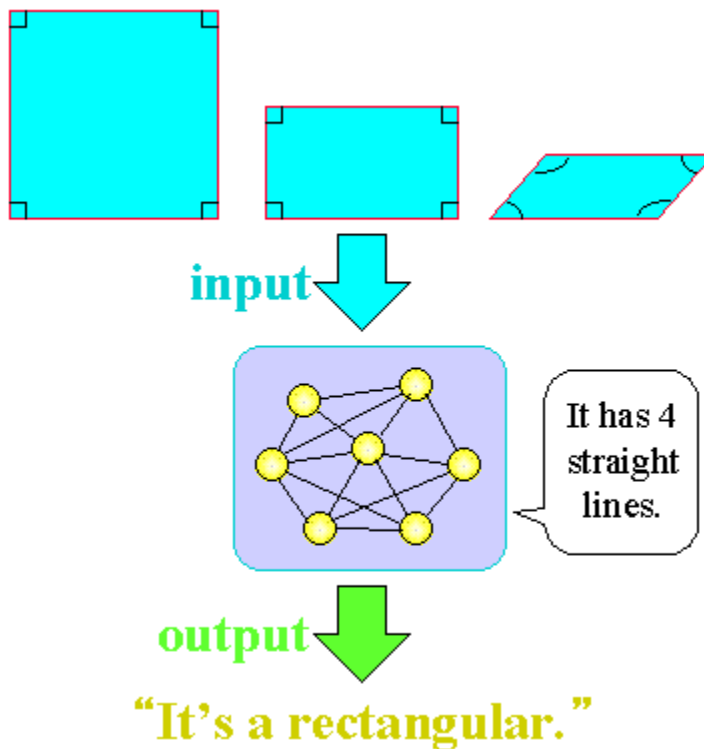
Supervised Learning

- In supervised learning, the network is presented with inputs together with the target (teacher signal) outputs. Then, the neural network tries to produce an output as close as possible to the target signal by adjusting the values of internal weights. The most common supervised learning method is the “error correction method”.
- Error correction method is used for networks which their neurons have discrete output functions. Neural networks are trained with this method in order to reduce the error (difference between the network's output and the desired output) to zero



Unsupervised Learning

- In unsupervised learning, there is no teacher (target signal) from outside and the network adjusts its weights in response to only the input patterns. A typical example of unsupervised learning is Hebbian learning



Consider a machine (or living organism) which receives some sequence of inputs $x_1, x_2, x_3, \dots$, where x_t is the sensory input at time t . In supervised learning the machine is given a sequence of input & a sequence of desired outputs $y_1, y_2, \dots$, and the goal of the machine is to learn to produce

the correct output given a new input. While, in unsupervised learning the machine simply receives inputs $x_1, x_2, \dots$, but obtains neither supervised target outputs, nor rewards from its environment. It may seem somewhat mysterious to imagine what the machine could possibly learn given that it doesn't get any feedback from its environment. However, it is possible to develop of formal framework for unsupervised learning based on the notion that the machine's goal is to build representations of the input that can be used for decision making, predicting future inputs, efficiently communicating the inputs to another machine, etc. In a sense, unsupervised learning can be thought of as finding patterns in the data above and beyond what would be considered pure unstructured noise.

Hebbian Learning

The oldest and most famous of all learning rules is Hebb's postulate of learning:

—When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic changes take place in one or both cells such that A's efficiency as one of the cells firing B is increased.

From the point of view of artificial neurons and artificial neural networks, Hebb's principle can be described as a method of determining how to alter the weights between model neurons. **The weight between two neurons increases if the two neurons activate simultaneously—and reduces if they activate separately.** Nodes that tend to be either both positive or both negative at the same time have strong positive weights, while those that tend to be opposite have strong negative weights

Hebbian learning is one of the oldest learning algorithms, and is based in large part on the dynamics of biological systems. A synapse between two neurons is strengthened when the neurons on either side of the synapse (input and output) have highly correlated outputs.

“When neuron A repeatedly and persistently takes part in exciting neuron B, the synaptic connection from A to B will be strengthened.”

Simultaneous activation of neurons leads to pronounced increases in synaptic strength between them. In other words, **"Neurons that fire together, wire together"**.

From the above postulate, we can conclude that the connections between two neurons might be strengthened if the neurons fire at the same time and might weaken if they fire at different times.

Mathematical Formulation – According to Hebbian learning rule, following is the formula to increase the weight of connection at every time step.

$$\Delta w_{ji}(t) = \alpha x_i(t) \cdot y_j(t)$$

Here,

$\Delta w_{ji}(t)$ = increment by which the weight of connection increases at time step t

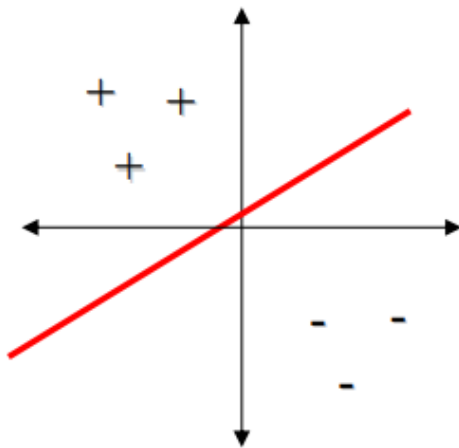
α = the positive and constant learning rate

$x_i(t)$ = the input value from pre-synaptic neuron at time step t

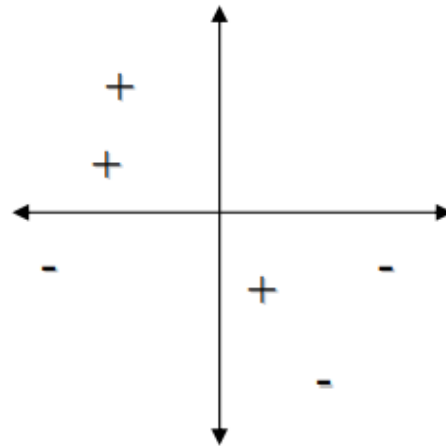
$y_j(t)$ = the output of post-synaptic neuron at same time step t

Perceptron Learning Rule

- The term "Perceptrons" was coined by Frank Rosenblatt in 1962 and is used to describe the connection of simple neurons into networks. These networks are simplified versions of the real nervous system where some properties are exaggerated and others are ignored. For the moment we will concentrate on Single Layer Perceptron.
- So how can we achieve learning in our model neuron? We need to train them so they can do things that are useful. To do this we must allow the neuron to learn from its mistakes. There is in fact a learning paradigm that achieves this, it is known as supervised learning and works in the following manner.
 - i. set the weight and thresholds of the neuron to random values.
 - ii. present an input.
 - iii. calculate the output of the neuron.
 - iv. alter the weights to reinforce correct decisions and discourage wrong decisions, hence reducing the error. So for the network to learn we shall increase the weights on the active inputs when we want the output to be active, and to decrease them when we want the output to be inactive.
 - v. Now present the next input and repeat steps iii. - v.
- A perceptron can learn only examples that are called “linearly separable”
- Learning a perceptron means finding the right values



Linearly separable



Non-linearly separable

There are two popular weight update rules in perceptron learning. They are:

- i) The perceptron rule, and
- ii) Delta rule

The Perceptron Rule

For a new training example $X = (x_0, x_2, \dots, x_n)$, update each weight according to this rule:

$$w_i = w_i + \Delta w_i$$

Where

$$\Delta w_i = \eta (t - o) x_i$$

t : target output

o : output generated by the perceptron

η : constant called the learning rate (e.g., 0.1)

Algorithm:

i. Initialize weights and threshold.

Set $w_i(t)$, ($0 \leq i \leq n$), to be the weight i at time t , and θ to be the threshold value in the output node. Set w_0 to be $-\theta$, the bias, and x_0 to be always 1.

Set $w_i(0)$ to small random values, thus initializing the weights and threshold.

ii. Present input and desired output

Present input $x_0, x_1, x_2, \dots, x_n$ and desired output $d(t)$

iii. Calculate the actual output

$$y(t) = g [w_0(t)x_0(t) + w_1(t)x_1(t) + \dots + w_n(t)x_n(t)]$$

iv. Adapt the weights

$w_i(t+1) = w_i(t) + \alpha[d(t) - y(t)]x_i(t)$, where $0 \leq \alpha \leq 1$ (learning rate) is a positive gain function that controls the adaption rate.

Steps iii. and iv. are repeated until the iteration error is less than a user-specified error threshold or a predetermined number of iterations have been completed.

Please note that the weights only change if an error is made and hence this is only when learning shall occur.

Delta Rule

What happens if the examples are not linearly separable? To address this situation we try to approximate the real concept using the delta rule. The key idea is to use a gradient descent search. Delta rule is a gradient descent learning rule for updating the weights of the inputs to artificial neurons in a single-layer neural network. It is a special case of the more general backpropagation algorithm.

For a neuron j with activation function $g(x)$ the delta rule for j 's i th weight w_{ji} is given by

$$\Delta w_{ji} = \alpha(t_j - y_j)g'(h_j)x_i$$

Where,

α is learning rate

t_j is target output

$g(x)$ is neuron's activation function

g' is derivative of g

h_j is weighted sum of neuron's input

y_j is actual output

x_i is the i th input

It holds that $h_j = \sum x_i w_{ji}$ and $y_j = g(h_j)$

The delta rule is commonly stated in simplified form for a neuron with a linear activation function as

$$\Delta w_{ji} = \alpha(t_j - y_j)x_i$$

The delta rule is derived by attempting to minimize the error in the output of the neural network through gradient descent. The error for a neural network with j outputs can be measured as

$$E = \sum_j \frac{1}{2} (t_j - y_j)^2.$$

Back propagation learning

It is a supervised learning method, and is an implementation of the Delta rule. It requires a teacher that knows, or can calculate, the desired output for any given input. It is most useful for feed-forward networks (networks that have no feedback, or simply, that have no connections that loop). The term is an abbreviation for "backwards propagation of errors". Backpropagation requires that the activation function used by the artificial neurons (or "nodes") is differentiable.

As the algorithm's name implies, the errors (and therefore the learning) propagate backwards from the output nodes to the inner nodes. So technically speaking, backpropagation is used to calculate the gradient of the error of the network with respect to the network's modifiable weights. This gradient is almost always then used in a simple stochastic gradient descent algorithm, is a general optimization algorithm, but is typically used to fit the parameters of a machine learning model, to find weights that minimize the error. Often the term "backpropagation" is used in a more general sense, to refer to the entire procedure encompassing both the calculation of the gradient and its use in stochastic gradient descent. Backpropagation usually allows quick convergence on satisfactory local minima for error in the kind of networks to which it is suited.

Backpropagation networks are necessarily multilayer perceptrons (usually with one input, one hidden, and one output layer). In order for the hidden layer to serve any useful function, multilayer networks must have non-linear activation functions for the multiple layers: a multilayer network using only linear activation functions is equivalent to some single layer, linear network.

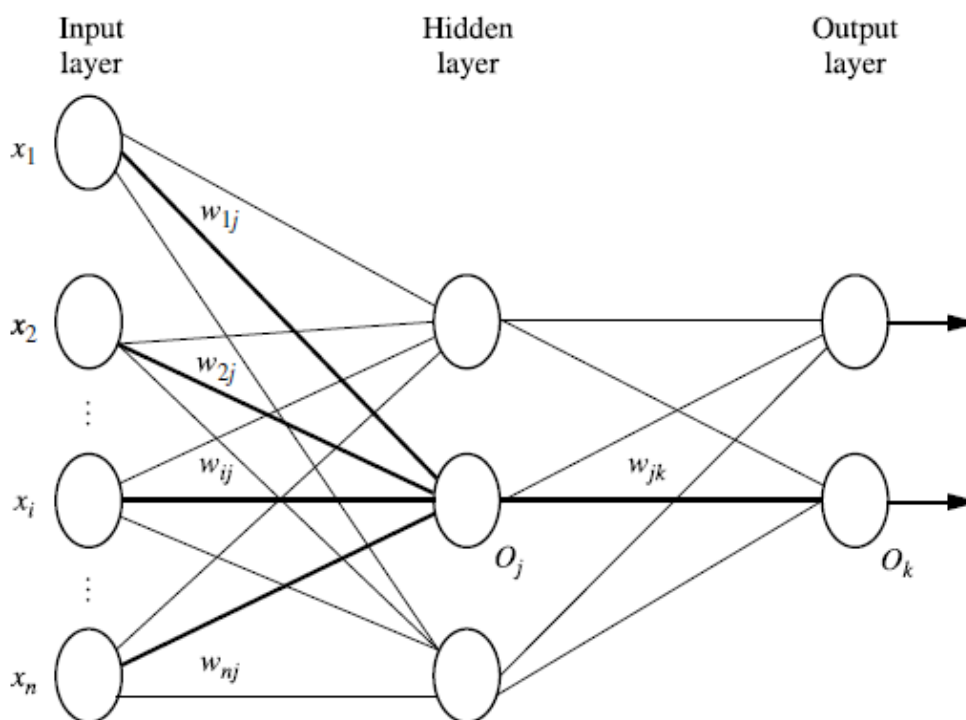
- Back-propagation is the essence of neural net training. It is the practice of fine-tuning the weights of a neural net based on the error rate (i.e. loss) obtained in the previous epoch (i.e. iteration). Proper tuning of the weights ensures lower error rates, making the model reliable by increasing its generalization.

Summary of the backpropagation technique:

- Present a training sample to the neural network.
- Compare the network's output to the desired output from that sample. Calculate the error in each output neuron.
- For each neuron, calculate what the output should have been, and a scaling factor, how much lower or higher the output must be adjusted to match the desired output. This is the local error.
- Adjust the weights of each neuron to lower the local error.
- Assign "blame" for the local error to neurons at the previous level, giving greater responsibility to neurons connected by stronger weights.
- Repeat from step 3 on the neurons at the previous level, using each one's "blame" as its error.

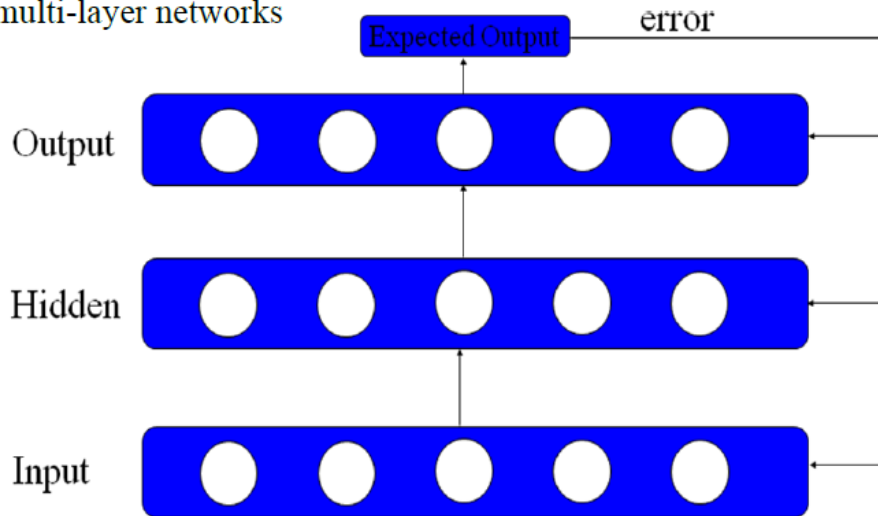
The **backpropagation algorithm** performs learning on a **multilayer feed-forward neural network**. It iteratively learns a set of weights for prediction of the class label of tuples.

A multilayer feed-forward neural network consists of an **input layer**, one or more **hidden layers**, and an **output layer**.



Backpropagation learns by iteratively processing a data set of training tuples, comparing the network's prediction for each tuple with the actual known target value. The target value may be the known class label of the training tuple (for classification problems) or a continuous value (for numeric prediction). For each training tuple, the weights are modified so as to minimize the mean-squared error between the network's prediction and the actual target value. These modifications are made in the "backwards" direction (i.e., from the output layer) through each hidden layer down to the first hidden layer (hence the name backpropagation). Although it is not guaranteed, in general the weights will eventually converge, and the learning process stops.

The backpropagation algorithm provides a computationally efficient method for training multi-layer networks



Main Steps in Back propagation algorithm

1. Initialization of weights
2. Feed forward
3. Back propagation of error
4. Updating of weights and biases

Algorithm:

Step 0: Initialize the weights to small random values

Step 1: Feed the training sample through the network and determine the final output

Step 2: Compute the error for each output unit, for unit k it is:

$$\delta_k = (t_k - y_k) f'(y_{in_k})$$

Actual output (points to y_k)
Required output (points to t_k)
Derivative of f (points to f')

Step 3: Calculate the weight correction term for each output unit, for unit k it is:

$$\Delta w_{jk} = \alpha \delta_k z_j$$

A small constant Hidden layer signal

Step 4: Propagate the delta terms (errors) back through the weights of the hidden units where the delta input for the j^{th} hidden unit is:

$$\delta_{in_j} = \sum_{k=1}^m \delta_k w_{jk}$$

The delta term for j^{th} hidden unit is:

$$\delta_j = \delta_{in_j} f'(z_{in_j})$$

Step 5: Calculate the weight correction term for the hidden units:

$$\Delta w_{ij} = \alpha \delta_j x_i$$

Step 6: Update the weights

$$w_{jk}(\text{new}) = w_{jk}(\text{old}) + \Delta w_{jk}$$

Step 7: Test for stopping (maximum cycles, small changes, etc) and repeat from step 1 if not terminated.